

Project Documentation: Technical Specification

1. Executive Summary

The **Attendance Calculator & Resource Scheduler** is an enterprise-grade backend solution built on **Spring Boot 3** and **Java 17**. It addresses the complexities of workforce scheduling, real-time availability tracking, and conflict-free resource booking. The system is designed to handle high-throughput data ingestion (CSV) and provide real-time analytics for management.

2. Technology Stack

The platform leverages a modern, high-performance stack to ensure scalability and ease of deployment:

- **Framework:** Spring Boot 3.x for rapid application development and dependency management.
- **Language:** Java 17 (LTS), utilizing records, sealed classes, and enhanced switch expressions for cleaner code.
- **Persistence:** Spring Data JPA with Hibernate as the ORM provider.
- **Database:** PostgreSQL (recommended) or any RDBMS for robust relational data storage and ACID compliance.

3. Architectural Design

The system follows a classic **N-Tier Layered Architecture**, ensuring separation of concerns and maintainability:

- **Controller Layer:** RESTful endpoints with request validation.
- **Service Layer:** Contains core business logic, including the unique 15-minute slot bitmasking algorithm.
- **Repository Layer:** Leverages Spring Data JPA with optimized JPQL queries (using **JOIN FETCH** to mitigate N+1 query issues).

4. Key Features

The system provides critical functionalities for efficient resource management:

- **Advanced Booking Management:** Supports creating, updating, and cancelling resource bookings with automated overlap detection.

- **Real-time Availability Tracking:** Instant lookups for employee or room availability based on current booking state.
- **High-Speed CSV Ingestion:** Bulk upload employee and schedule data via optimized multi-row processing.

5. Core Logic: The 15-Minute Slot Engine

One of the key technical innovations is the `AvailabilityService` logic:

- **Slot Indexing:** Instead of repetitive database checks for every minute, the system maps a requested time window into discrete 15-minute indices.
- **Bitmask-style Boolean Arrays:** For every employee, a boolean array `unavailableSlots[]` is generated. Existing bookings are "projected" onto this array as `true` values in $O(1)$ time per booking.
- **Final Tally:** A single pass over the array determines availability, allowing the system to scale efficiently even as the number of employees or bookings grows.

6. Performance & Data Integrity

- **Transactional Integrity:** All booking operations are `@Transactional`, ensuring that "double-booking" is logically impossible during concurrent requests.
- **Batch Ingestion:** The `EmployeeCsvService` employs batch processing. By fetching existing records in a single `findAllById` call before processing, we reduce database round-trips from $O(N)$ to $O(1)$.
- **Memory Management:** Heavy computations (like attendance summaries) are handled via Java Streams and `Collectors.groupingBy`, reducing heap pressure by avoiding manual map manipulations.

7. API Endpoints

The following RESTful endpoints provide the core interface for the application:

Endpoint	Method	Description
/api/bookings	POST	Creates a new resource booking.
/api/availability	GET	Queries availability for a specific time window.

Endpoint	Method	Description
/api/employees/upload	POST	Uploads employee data via CSV multipart file.

8. Database Schema

The relational model is designed for consistency and quick indexing:

- **Employee:** Stores core profiles (name, shift, department).
- **Booking:** Links employees to specific time slots and resources.
- **Slot:** Represents discrete 15-minute intervals used for the bitmasking logic.

9. Future Roadmap

The following enhancements are planned for future iterations:

- **API Documentation:** Integration of Swagger/OpenAPI for interactive endpoint testing.
- **Containerization:** Full Docker support for consistent environments across development and production.
- **Cloud Deployment:** CI/CD pipelines for automated deployment to AWS or Azure.